ADVANCED DATA STRUCTURES LABORATORY
IT5412

A MINI PROJECT REPORT

Submitted By

Niyathi S B	2022506077
Yuvaraj K	2022506079
Aadhira R	2022506087

DEPARTMENT OF INFORMATION TECHNOLOGY

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY, CHENNAI – 600 044

JANUARY – MAY 2024

Problem Description

Huffman coding is a greedy-strategy or a greedy-algorithm which is used in data compression. It maintains data about each distinct character that is present in a file and its frequency, which is the number of times that it appears in the file. Each such character is represented by a *variable-length, binary character code,* which are called as Huffman codes.

These Huffman codes are formed for each character in such a way that

- the more frequently appearing characters are given shorter code words
- the less frequently appearing characters are given longer code words

The beautiful thing about Huffman coding is that these code words are all perfectly differentiable from each other. That is, given any string containing multiple code words, it parses uniquely to give back the original string. Huffman coding is thus an easy-to-implement and efficient way of data compression, encoding, and decoding.

```
HUFFMAN (C)
    // C – set of n characters, each c has c.freq
    n = |C|
    Q = C
    for i = 1 to n-1
        allocate a new node z
        x = EXTRACT-MIN (Q)
        y = EXTRACT-MIN (Q)
        z.left = x
        z.right = y
        z.freq = x.freq + y.freq
        INSERT (Q, z)
    return EXTRACT-MIN (Q) // only root is left
```

Code Structure

File Name	Purpose
headers.hpp	All header files to be included.
main.cpp	The main program that is to be executed. Has options for encoding and decoding, and based on the chosen option, navigates to the respective file functions and executes them.
encode.hpp	Function declarations for encode.cpp.
encode.cpp	Computes Huffman codes, builds the tree, and stores the codes and compressed data in separate files.
decode.hpp	Function declarations for decode.cpp
decode.cpp	Rebuilds the original file contents based on the Huffman codes and compressed file.
sample.txt	Sample text file to showcase the encoding and decoding using Huffman coding.

Sample Input

File Name	*sample.txt*
Contents	*This is a sample file.*

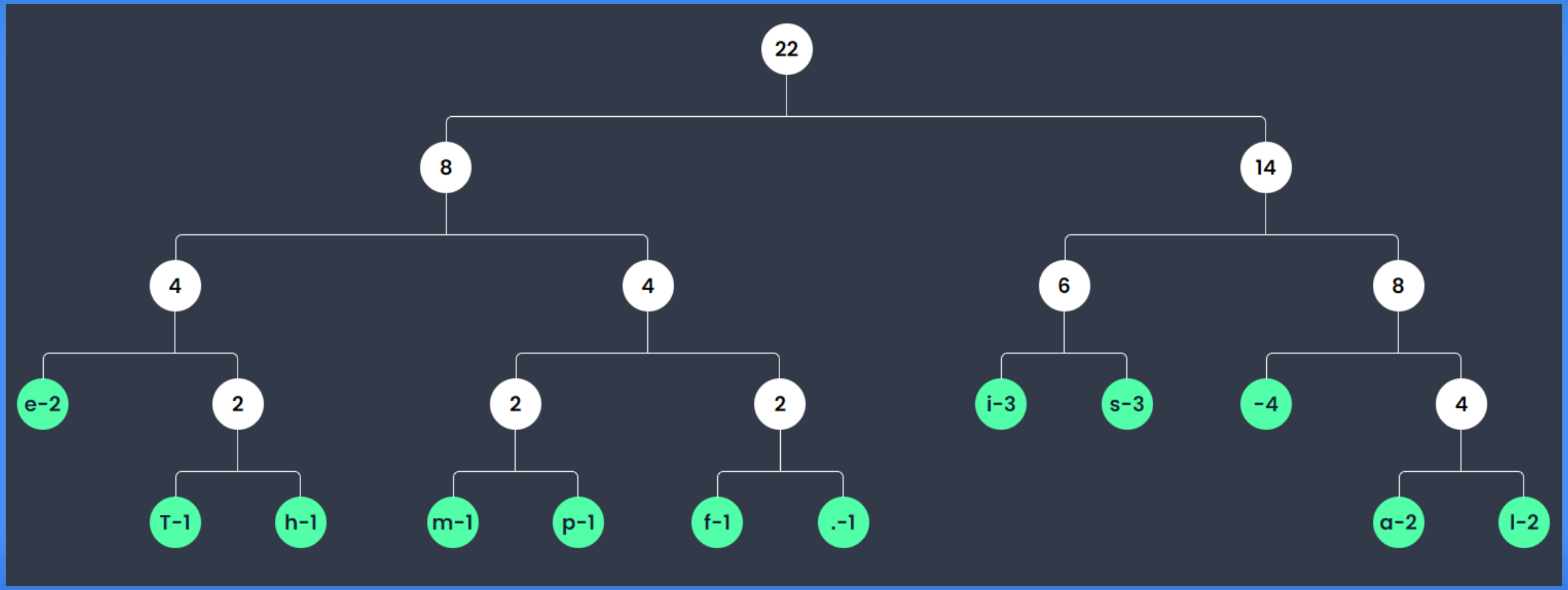
The characters and their corresponding Huffman codes are stored in both the *codes.txt* and the *codes.dat* file. The *codes.dat* file is there for our convenience – to read the codes in an easier manner. The compressed file, which contains the string of Huffman codes representing the contents of the original file, is stored in both the *compressed.txt* file and the *compressed.dat* file.

While decoding to get back the original file, both the *codes.dat* and the *compressed.dat* have to be given as input.

Frequency Table

Character	Frequency	Huffman Code
T	1	0010
h	1	0011
i	3	100
s	3	101
[space]	4	110
a	2	1110
m	1	0100
p	1	0101
l	2	1111
e	2	000
f	1	0110
.	1	0111

Generated Huffman Tree



Source: Huffman Coding

Challenges Faced

Segmentation Faults	Having to deal with a lot of arrays, static and dynamic memory allocations, file reading and file writing, we knew first and foremost going in that we were going to face many segmentation faults, several errors with regards to how we implemented our memory handling. It was difficult to find where exactly we had made such errors. In some cases, we were able to correctly and logically point out the exact reason for the segmentation faults, while others took a lot of tiresome trail and error.
File Handling	We were all familiar and quite well-versed with the various file handling mechanisms in C, and while we knew that C++ had its own file handling headers and operations, we weren't sure if we could be as versatile with them. As such, we made a calculated decision to go ahead and use C file handling for our project. We are lucky that C++ supports C file handling as well, which not only speaks to its effectiveness, but also to its compatibility to the many things that C already offers.
Decompression	While the encoding and compression was relatively straightforward, involving frequency calculations and tree constructions, the decoding and decompression was a completely different story. The words after a space weren't being decoded properly. Some particular letters weren't being decoded in the expected manner. Figuring out why this happened and fixing them was certainly difficult.
Communication	This implies two types of communication - ensuring that the different program and text files communicated properly with each other, and enabling proper communication amongst three programmers with wildly varying programming styles. Both of these were equally challenging, and it took a lot of time and effort to achieve a sense of proper synchronisation.

Individual Contributions

While there was undeniable overlap and contributions in all aspects of the code by every individual for this project, there was a *head* or say, a *leading programmer* for each particular area or section of the code, which is what is highlighted below. This can be considered as the primary contribution of the member, who also most certainly worked on other parts of the project and aided in every way possible.

Member	Primary Contribution	Description
Niyathi S B	Encoding	Computing the frequency table and generating the Huffman codes; storing the aforementioned in the required files such that they could be used later
Aadhira R	Decoding	Regenerating the original file using the Huffman codes and compressed data, parsing the string uniquely so as to get the original data accurately
Yuvaraj K	Testing and Debugging	Ensuring that the code worked perfectly for different inputs, and making changes to fix any logical error that was encountered

The code-wise communication for this project was done through a private, shared repository in Github. Here, we added new test files, made changes to the existing code, and committed/saved the approved modifications in order to eventually arrive at the final, finished version of the project.

Result

After a few weeks of hard work, encoding, decoding, and testing, we were able to successfully arrive at a stable version of Huffman coding. There could be many more improvements that could be made, certainly. Maybe some way to graphically and colourfully represent our outputs, maybe a more interactive user interface, and so on. But for now, we have showcased a thorough conceptual and practical understanding of how to implement Huffman coding in the C++ environment.

References

Reference	Purpose	Link (if any)
An Introduction to Algorithms, by Thomas Cormen	Huffman Coding Algorithm	-
Splay Tree – GeeksforGeeks	Understanding Splay Trees	*Splay Tree - GFG*
Splay Tree – Programiz	Understanding Splay Trees	*Splay Tree - Scaler*
Min Heaps - GeeksforGeeks	Understanding Min Heaps	*Min Heap - GFG*
Huffman Coding – Suhaan Bhandary	Verifying and Testing Huffman Codes	*Huffman Coding*